data['column_name'].astype('float') Similarly, converting a date column to a datetime type: data['date_column'] = pd.to_datetime(data['date_column']) Correct data types enable efficient storage and faster computations.

Normalizing and Scaling Data

Normalization and scaling are preprocessing techniques that ensure features are within a comparable range, which is critical for many machine learning algorithms. Python's Scikit-Learn library provides tools for both.

Standard Scaling

Standard scaling transforms features to have a mean of 0 and a standard deviation of 1: from sklearn.preprocessing import StandardScaler

```
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)
```

Min-Max Scaling

Min-max scaling rescales features to a fixed range, typically [0, 1]: from sklearn.preprocessing import MinMaxScaler

```
scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data)
```

Robust Scaling

Robust scaling is useful for datasets with outliers, as it scales features using the median and interquartile range: from sklearn.preprocessing import RobustScaler

```
scaler = RobustScaler()
data_scaled = scaler.fit_transform(data)
```

Encoding Categorical Variables

Machine learning models typically require numeric inputs, so categorical variables need to be encoded.

Common methods include:

- **Label Encoding:** Assigns a unique integer to each category:

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
data['encoded_column'] = encoder.fit_transform(data['categorical_column'])
```